

MULTI-TIER ARCHITECTURE USING AWS FOR CLOUD-BASED DIGITAL LIBRARY SYSTEM

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

AKULA SIVA NAGA PRASAD, 23221A0506, BVC ENGINEERING COLLEGE (Batch-1)

CHITNEEDI VEERA NAGA SATYANARAYANA, 23221A0528, BVC ENGINEERING COLLEGE (Batch-1)

CHAPPIDI GANESH, 23221A0526, BVC ENGINEERING COLLEGE (Batch-1)

JAGATHA BHARGAV SATISH, 23221A0545, BVC ENGINEERING COLLEGE (Batch-1)

ADDAGARALA SATYA CHANDRA HANUMA, 23221A0503, BVC ENGINEERING COLLEGE (Batch-1)

BANDIGUPTAPU SATYA VARA PRASAD, 23221A0517, BVC ENGINEERING COLLEGE (Batch-1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to **Mrs Sumana Bethala** my supervisor and mentor, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

A.S.N.prasad,23221A0506, shivaakula90492@gmail.com
Ch.V.N.Satyanarayana,23221A0528,chitnidiveeranagasatyanarayana@gmail.com
ChappidiGanesh,23221A0526,ganeshchappidi708@gmail.com
J.B.Satish,23221A0545,bhargavjagatha99@gmail.com
A.S.C.Hanuma,23221A0503,chanduhanuma5@gmail.com
B.S.V.Prasad,23221A0517,bsvprasad630@gmail.com

ABSTRACT

This project presents the implementation of a **Cloud-Based Digital Library System** using **Amazon Web Services (AWS)** to provide a secure, scalable, and efficient platform for managing digital books. The primary goal is to leverage cloud computing technologies to build a reliable, web-based solution that enables centralized storage, efficient management, and easy access to digital learning resources while ensuring secure data storage and high system availability.

The system workflow begins with the deployment of a **Flask-based web application** on an **Amazon EC2** instance within a secure **Virtual Private Cloud (VPC)**. Administrators can upload PDF books along with details such as title, author, and category through the application. The uploaded files are securely stored in **Amazon S3**, while the corresponding metadata is maintained in **Amazon DynamoDB** for efficient retrieval and search operations. Users can browse the available books, search using various criteria, and download the required documents through a simple and user-friendly interface.

This architecture utilizes **Amazon EC2** for application hosting, **Amazon VPC** for secure networking, **Amazon S3** for file storage, **Amazon DynamoDB** for real-time database management, and **Amazon CloudWatch** for monitoring application performance and resource utilization. The solution provides improved scalability, centralized storage, simplified management, and reliable access to digital resources without requiring complex infrastructure management.

By integrating multiple AWS services in a multi-tier architecture, the system enables secure and efficient digital book management. It improves resource availability, operational efficiency, and demonstrates the practical use of AWS cloud technologies, while providing a foundation for future enhancements such as user authentication, role-based access control, CI/CD integration, and mobile support.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	5
2.	METHODOLOGY	6
3.	TECHNOLOGY STACK	8
4.	SYSTEM DESIGN / ARCHITECTURE.....	10
5.	IMPLEMENTATION	17
6.	RESULTS	19
7.	CONCLUSION	23
8.	REFERENCES.....	24

1. INTRODUCTION

In the digital era, where cloud computing, secure data storage, and easy access to information have become essential, digital library systems play a significant role in modern education and knowledge management. This project, titled **Cloud-Based Digital Library System**, utilizes the power of **Amazon Web Services (AWS)** to develop a secure, scalable, and efficient platform for managing digital books. It demonstrates how cloud technologies can simplify traditional library operations by providing centralized storage, faster access, and reliable resource management through a web-based application.

Educational institutions and organizations require uninterrupted access to digital resources, efficient search functionality, and secure storage for books and study materials. Traditional library systems often depend on manual record maintenance or local storage, making them difficult to manage and expand. This project addresses these challenges by adopting a multi-tier cloud architecture, where Amazon EC2 hosts the Flask web application, Amazon S3 stores PDF books, Amazon DynamoDB maintains book metadata, and Amazon VPC provides a secure networking environment. Users can upload, browse, search, download, and manage digital books through an easy-to-use interface, ensuring better accessibility and improved resource management.

Designed with simplicity, scalability, and reliability, the system allows administrators to upload PDF books along with details such as title, author, and category. The uploaded files are securely stored in Amazon S3, while their metadata is stored in Amazon DynamoDB for efficient retrieval and search operations. The application is deployed on an Amazon EC2 instance within a secure Virtual Private Cloud (VPC), and Amazon CloudWatch is used to monitor application performance and resource utilization. This architecture improves security, scalability, and overall system performance while reducing the limitations of traditional library management.

This report presents the design, architecture, implementation, deployment, and testing of the proposed system. It demonstrates how AWS cloud services and DevOps concepts can be effectively integrated to develop a practical, secure, and scalable digital library solution for educational institutions, libraries, and organizations.

2. METHODOLOGY

The development of the **Cloud-Based Digital Library System** followed a structured and systematic methodology to ensure the successful achievement of the project objectives. The methodology was divided into several phases, including requirements gathering, system design, implementation, database management, testing, deployment, and continuous refinement. Each phase was planned according to cloud computing best practices and multi-tier architecture principles using Amazon Web Services (AWS). The following subsections describe each phase in detail.

2.1. Requirements Gathering

The project began with a detailed analysis of the functional and technical requirements for developing a cloud-based digital library. Discussions with mentors and the study of existing library systems helped identify the major requirements, including secure book storage, metadata management, search functionality, and cloud-based deployment. The system was designed to allow administrators to upload digital books and enable users to browse, search, and download books through a simple web interface.

2.2. Design

Based on the identified requirements, a multi-tier cloud architecture was designed using AWS services. The architecture includes Amazon EC2 for hosting the Flask web application, Amazon VPC for secure networking, Amazon S3 for storing PDF books, Amazon DynamoDB for managing book metadata, and Amazon CloudWatch for monitoring system performance.

2.3. Implementation

The implementation phase focused on developing a fully functional web application using the Flask framework. The application was deployed on an Amazon EC2 instance within a Virtual Private Cloud. Administrators can upload PDF books along with details such as title, author, and category. The uploaded files are stored in Amazon S3, while the corresponding metadata is stored in Amazon DynamoDB. The application also provides functionality for browsing, searching, downloading, and managing books through a web interface.

2.4.Database Management

A NoSQL database approach was adopted using Amazon DynamoDB to store book information efficiently. The table was designed with attributes such as Book ID, Title, Author, Category, Upload Date, and S3 File Location. This structure enables fast retrieval of book information and seamless integration with the web application. Data consistency and integrity are maintained during every upload and update operation.

2.5. Testing

Various testing techniques were performed to validate the functionality and performance of the application. Functional testing verified book upload, search, browse, download, and delete operations. Integration testing ensured proper communication between EC2, S3, DynamoDB, and CloudWatch. User interface testing confirmed that the application provides a simple and responsive experience for administrators and users.

2.6. Deployment

The application was deployed on Amazon EC2 using the AWS Management Console. A secure networking environment was established using Amazon VPC, while PDF files were stored in Amazon S3 and metadata was maintained in Amazon DynamoDB. Amazon CloudWatch was used to monitor application performance and resource utilization. The deployment process ensured that the application was accessible, reliable, and easy to manage.

2.7. Iterative Refinement

Throughout the development process, continuous improvements were made based on testing results and mentor feedback. Enhancements were implemented to improve the user interface, optimize application performance, and strengthen error handling. The architecture and application workflow were periodically reviewed to ensure scalability, maintainability, and efficient utilization of AWS services. This iterative approach resulted in a stable and reliable cloud-based digital library system suitable for educational institutions and organizations.

3. TECHNOLOGY STACK

The Cloud-Based Digital Library System was developed using a modern cloud computing technology stack that supports secure application hosting, cloud storage, database management, and system monitoring. The following technologies and services were used throughout the implementation.

3.1. Cloud Platform

Amazon Web Services (AWS): The entire project is deployed on AWS, utilizing cloud services for application hosting, networking, storage, database management, and monitoring. AWS provides a secure, reliable, and scalable cloud environment that enables efficient resource management, high availability, and seamless integration of various services to support the Digital Library application.

3.2. AWS Services

- **Amazon EC2 (Elastic Compute Cloud):** Hosts the Flask-based web application and provides a scalable virtual server environment.
- **Amazon VPC (Virtual Private Cloud):** Provides a secure and isolated network environment for hosting the EC2 instance.
- **Amazon S3 (Simple Storage Service):** Stores PDF books and digital documents securely with high durability and availability.
- **Amazon DynamoDB:** A fully managed NoSQL database used to store book metadata such as title, author, category, upload date, and file location.

- **Amazon CloudWatch:** Monitors EC2 instance performance, application logs, and resource utilization for system monitoring and troubleshooting.

3.3. Programming & Web Technologies

- **Python3:** Used for developing the backend application using the Flask framework.
- **Flask:** A lightweight Python web framework used to develop the Digital Library web application.
- **Boto3SDK:** Python SDK used to interact with AWS services such as Amazon S3 and Amazon DynamoDB.
- **HTML,CSS,andBootstrap:** Used to design and develop the user interface of the application.

3.4. Data Format

- **PDF (Portable Document Format):** Used for storing digital books.
- **JSON (JavaScriptObject Notation):** Used for communication between the Flask application and AWS services.

3.5. Security & Access

- **AWS SecurityGroups:** Control inbound and outbound network traffic for the EC2 instance.
- **Amazon VPC:** Provides secure network isolation for the application and cloud resources.

Note: Since this project was implemented using an AWS Student Account, IAM role creation was restricted. Therefore, IAM roles were not used in this implementation.

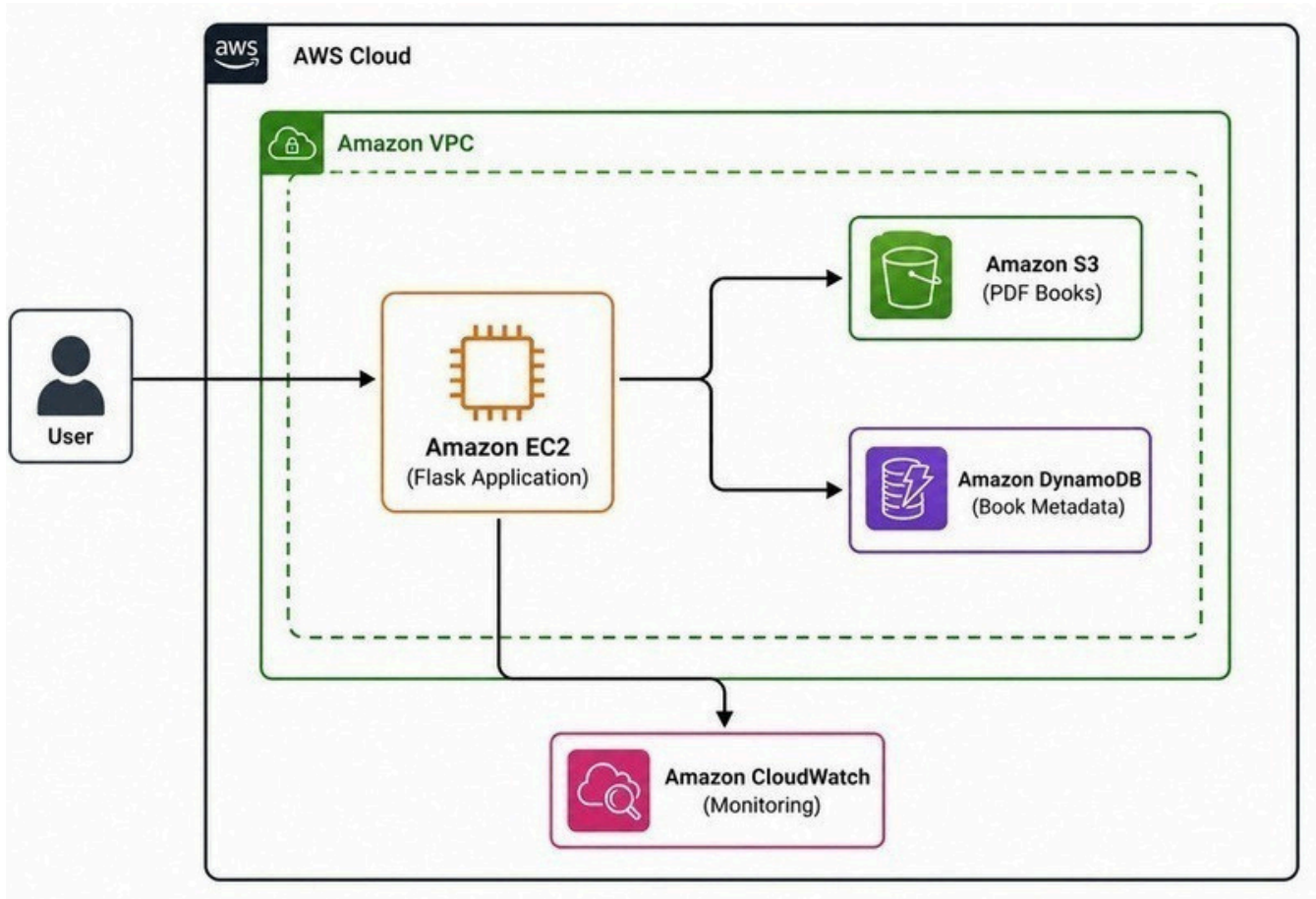
3.6. Monitoring & Logging

- **Amazon CloudWatch Metrics:** Monitors CPU utilization, network traffic, and EC2 instance performance.
- **Amazon CloudWatch Monitoring:** Provides real-time monitoring of application and infrastructure resources to ensure reliable system operation.

4. SYSTEM DESIGN / ARCHITECTURE

The Cloud-Based Digital Library System is designed as a multi-tier cloud application using Amazon Web Services (AWS). The system provides secure storage, efficient management, and easy access to digital books. The web application is hosted on Amazon EC2 within Amazon VPC, while Amazon S3 stores PDF books and Amazon DynamoDB stores book metadata. Amazon CloudWatch is used to monitor system performance and resource utilization.

The following architecture diagram illustrates the interaction between the AWS services used in the proposed Cloud-Based Digital Library System.



4.1. Network Layer – Amazon VPC

The **Amazon Virtual PrivateCloud (VPC)** provides a secure and isolated networking environment for the Cloud-Based Digital Library System. A custom VPC named **library-vpc** was created to host the AWS resources required for the application. The VPC uses the IPv4 CIDR block **10.0.0.0/16**, allowing secure communication between AWS services.

A **public subnet**, **route table**, and **Internet Gateway** were configured within the VPC to provide internet connectivity for the Amazon EC2 instance. This network architecture ensures secure deployment, controlled traffic flow, and reliable communication between the web application and AWS services.

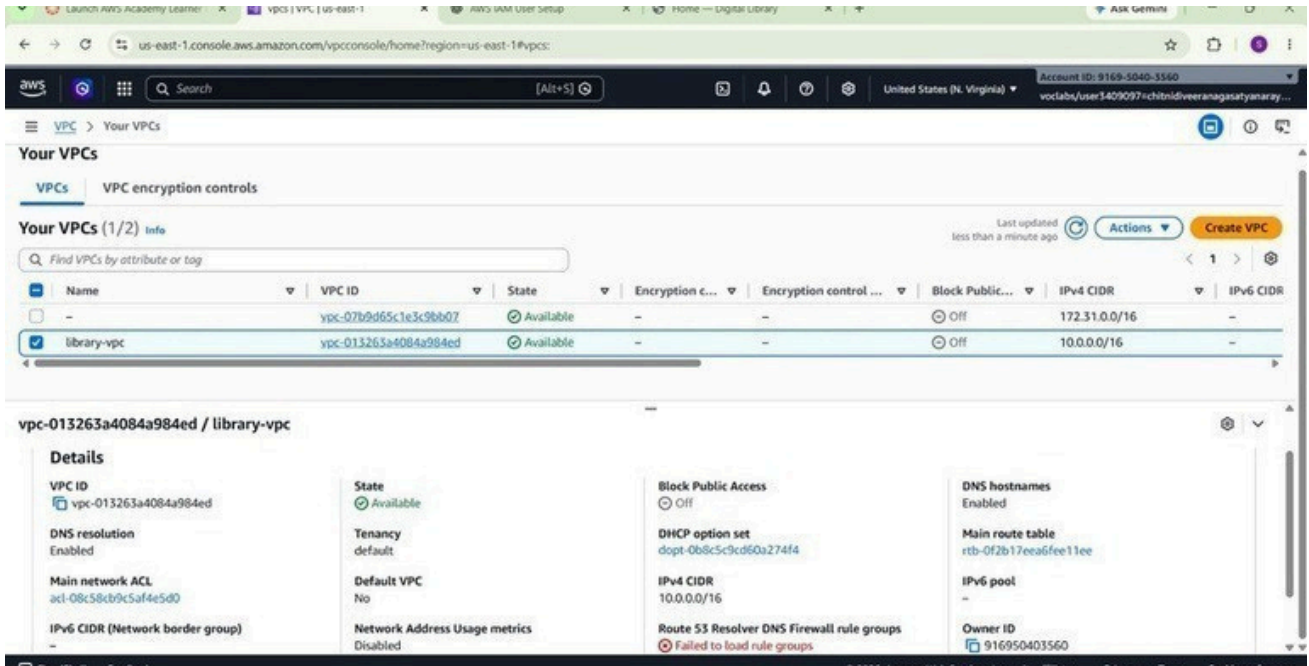


Figure 4.1: Amazon VPC (library-vpc) Configuration

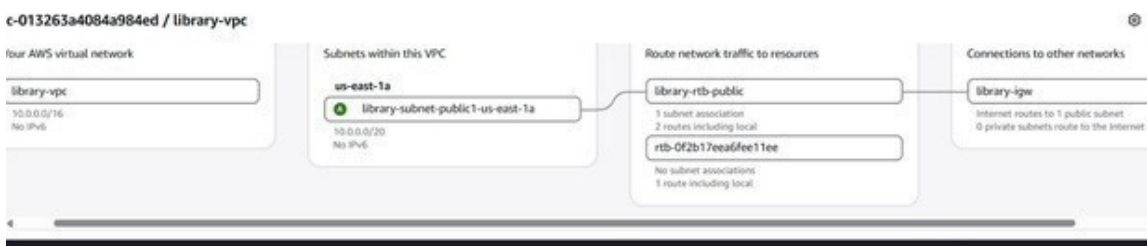


Figure 4.2: Amazon VPC Resource Map Showing Network Components

Key Features:

- Secure and isolated virtual network.
- Custom VPC with **10.0.0.0/16** CIDR block.
- Public subnet for EC2 deployment.
- Internet Gateway and Route Table for connectivity.

4.2. Compute Layer – Amazon EC2

AmaAmazonEC2(ElasticComputeCloud) is used to host the **Flask-based Digital Library** application. An EC2 instance named **library-web-server** was launched within the custom **Amazon VPC** to run the web application. The instance provides secure and reliable access to the Digital Library through a public IP address. It processes user requests such as uploading, browsing, searching, and downloading books. The EC2 instance communicates with **Amazon S3** for storing PDF files and **Amazon DynamoDB** for managing book metadata, ensuring efficient and seamless application performance.

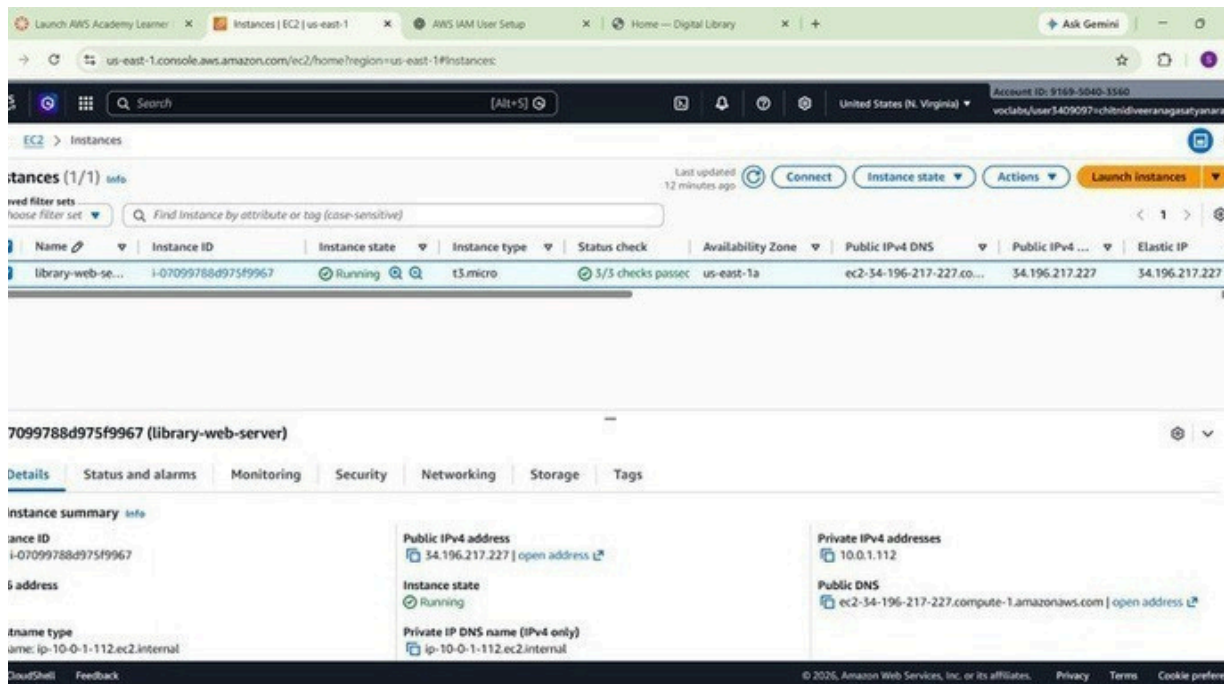


Figure 4.3: Amazon EC2 Instance (library-web-server) Overview



Figure 4.4: Amazon EC2 Instance Network Configuration

Key Features:

- Hosts the Flask web application.
- Runs inside the custom Amazon VPC.
- Provides public and private IP addresses.
- Ensures reliable and scalable application hosting.

4.3. Security Layer – Security Groups

AmazonEC2 Security Groups are used to protect the Digital Library application by controlling inbound and outbound network traffic. A custom security group named **library** was created and associated with the EC2 instance hosted inside the custom VPC. The security group acts as a virtual firewall, allowing only authorized traffic to access the application. It ensures secure communication between users and the web server while preventing unauthorized access to the instance. Appropriate rules were configured to support remote administration and web application access.

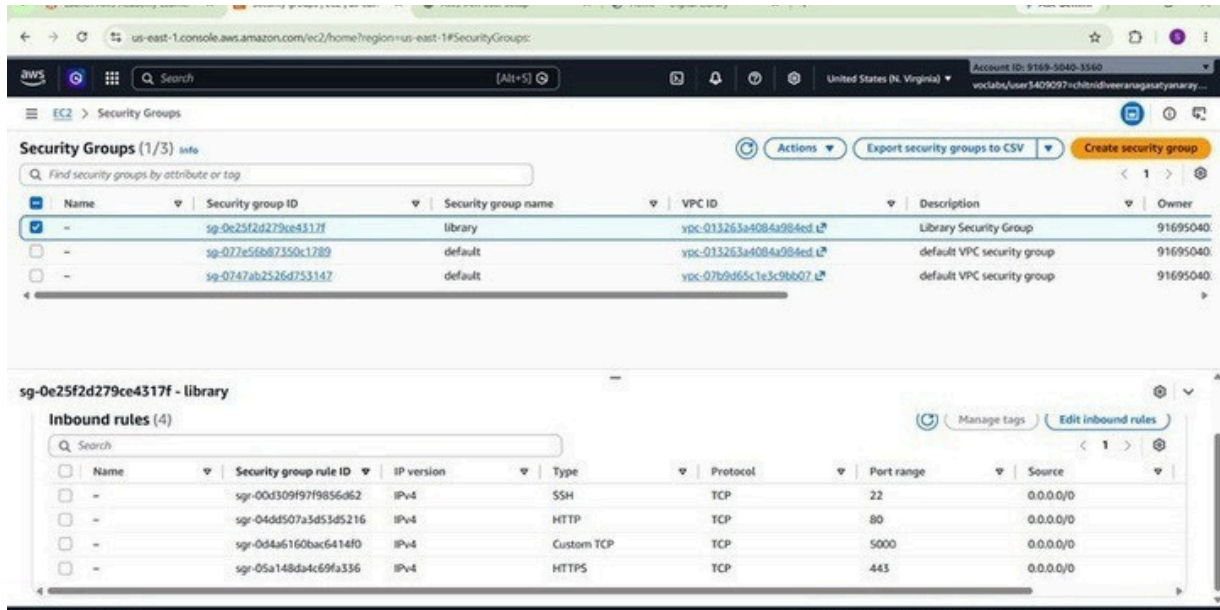


Figure 4.5: Custom Security Group (library) with Configured Inbound Rules

Key Features:

- Allows **SSH (Port 22)** for secure remote administration.
- Enables **HTTP (Port 80)** and **HTTPS (Port 443)** for web access.
- Allows **Custom TCP Port 5000** to access the Flask application.
- Filters incoming traffic to improve application security and network protection.

4.4.Storage Layer – Amazon S3

Amazon S3 (Simple Storage Service) is used to store the PDF books uploaded through the Digital Library application. A bucket named **digital-library-books-2026** was created to provide secure, scalable, and durable cloud storage. A dedicated **books/** folder is maintained inside the bucket to organize all uploaded PDF files. The Flask application uploads books to Amazon S3, and users can securely access and download them through the web interface.

Key Features:

- Stores uploaded PDF books securely.
- Provides scalable and highly durable cloud storage.
- Organizes files using the **books/** folder.
- Integrates with the Flask application for upload and download operations.

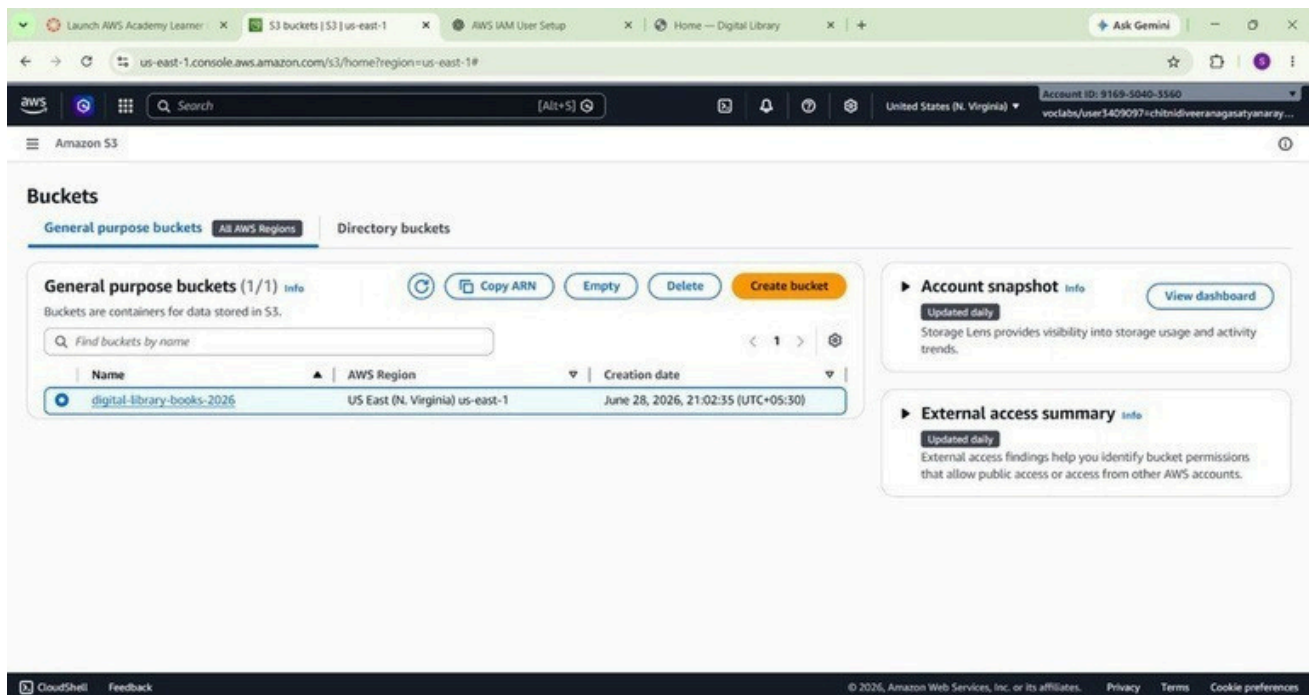


Figure 4.7: Amazon S3 Bucket (digital-library-books-2026)

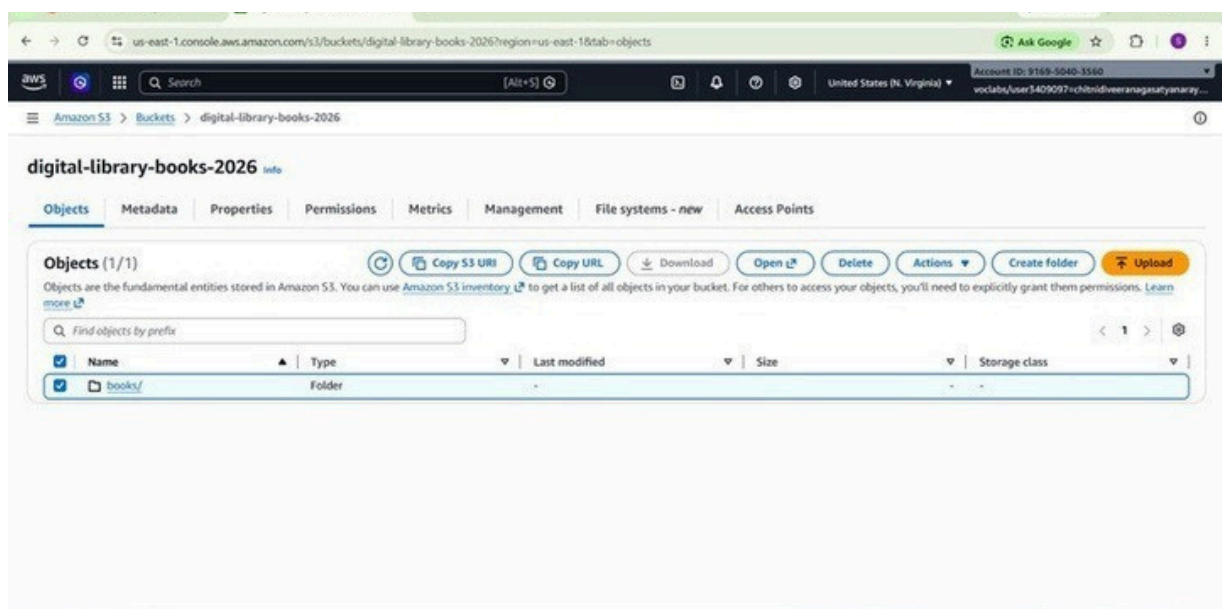


Figure 4.8: Books Folder Created in Amazon S3 Bucket

4.5. Database Layer – Amazon DynamoDB

Amazon DynamoDB is used to store the metadata of books uploaded to the Digital Library. A table named **BooksMetadata** was created with **book_id** as the partition key. The table stores important information such as the book title, author, category, S3 file path, and upload date. The Flask application interacts with DynamoDB to save, retrieve, search, and manage book records efficiently without storing the actual PDF files in the database.

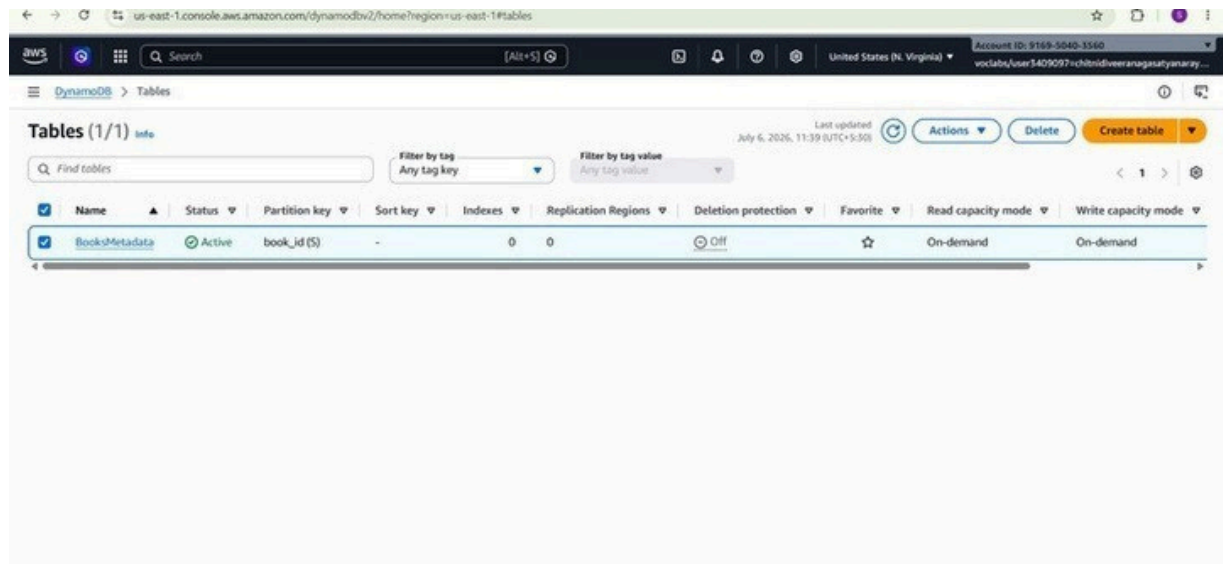


Figure 4.9: BooksMetadata Table in Amazon DynamoDB

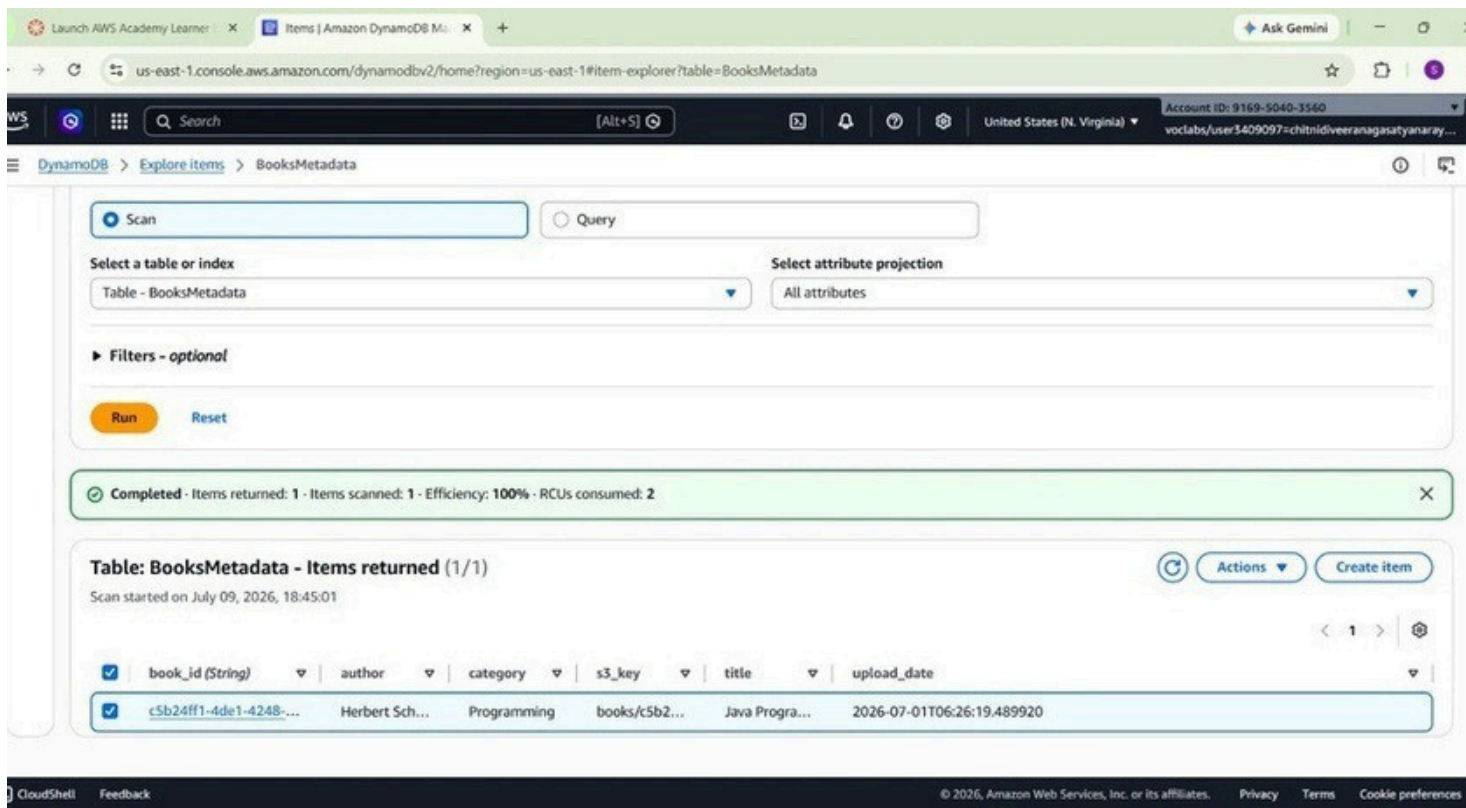


Figure 4.10: Book Metadata Stored in Amazon DynamoDB Table

Key Features:

- Stores book metadata using the **BooksMetadata** table.
- Uses **book_id** as the primary (partition) key.
- Supports fast search and retrieval of book information.
- Integrates with Amazon S3 to reference uploaded PDF files.

4.6. Monitoring and Logging–AmazonCloudWatch

Amazon CloudWatch is used to monitor the performance and health of the Digital Library application. It collects metrics, logs, and resource information from the EC2 instance, allowing administrators to monitor application activity and troubleshoot issues. CloudWatch helps ensure the web application remains available and performs efficiently.

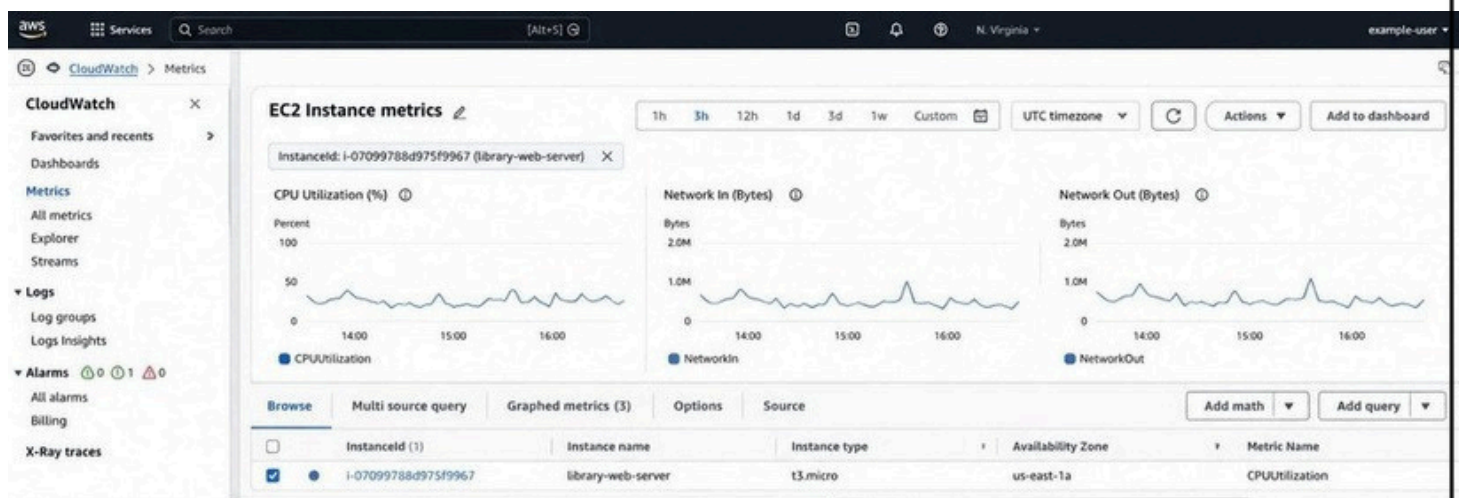


Figure 4.11: Amazon CloudWatch Metrics Dashboard

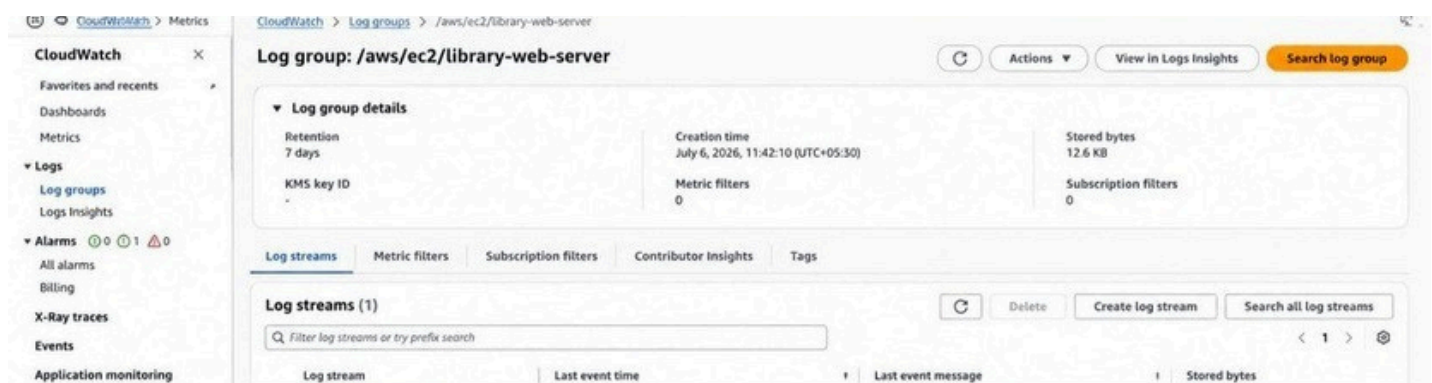


Figure 4.12: Amazon CloudWatch Logs for EC2 Instance

Key Features:

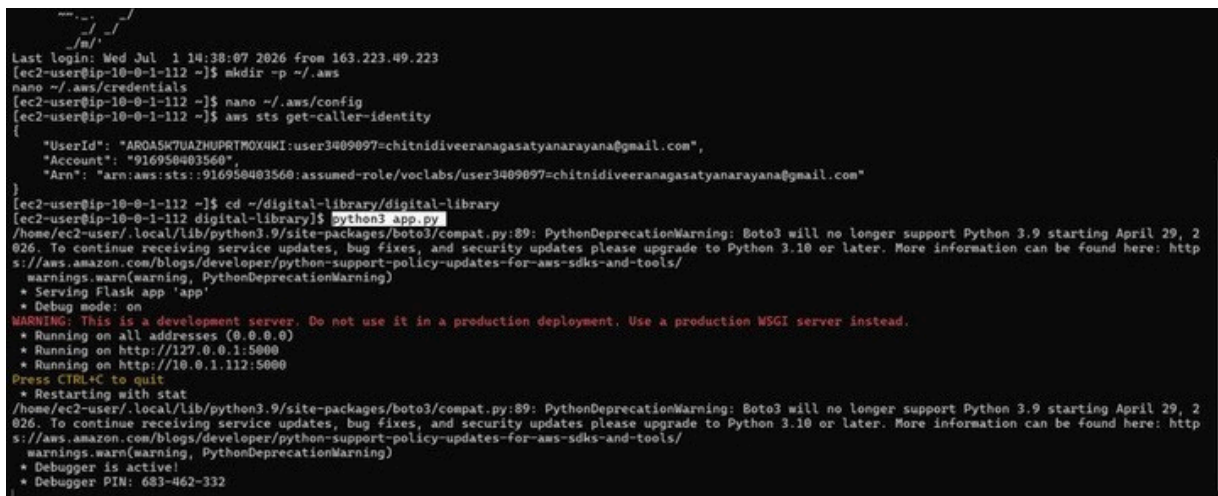
- Monitors EC2 instance performance.
- Collects application and system logs. Tracks
- CPU, network, and resource usage. Helps
- identify and troubleshoot issues.

5. IMPLEMENTATION

This chapter describes the implementation of the Cloud-Based Digital Library System using AWS cloud services and the Flask framework. The application was deployed on an Amazon EC2 instance and integrated with Amazon S3 and Amazon DynamoDB for storage and metadata management.

5.1 Environment Setup on Amazon EC2

The EC2 instance was configured with the required software to deploy the Flask application. Python, pip, Git, AWS CLI, Flask, Boto3, and Gunicorn were installed to support application development and communication with AWS services.



```
Last login: Wed Jul 1 14:38:07 2026 from 163.223.49.223
[ec2-user@ip-10-0-1-112 ~]$ mkdir -p ~/.aws
nano ~/.aws/credentials
[ec2-user@ip-10-0-1-112 ~]$ nano ~/.aws/config
[ec2-user@ip-10-0-1-112 ~]$ aws sts get-caller-identity
{
  "UserId": "AROASK7UAZHUPRTMOX4KI:user3409097=chitniveeranagasatyanarayana@gmail.com",
  "Account": "916950403560",
  "Arn": "arn:aws:sts::916950403560:assumed-role/voclabs/user3409097=chitniveeranagasatyanarayana@gmail.com"
}
[ec2-user@ip-10-0-1-112 ~]$ cd ~/digital-library/digital-library
[ec2-user@ip-10-0-1-112 digital-library]$ python3 app.py
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.1.112:5000
Press CTRL+C to quit
 * Restarting with stat
/home/ec2-user/.local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
 * Debugger is active!
 * Debugger PIN: 683-462-332
```

Figure 5.1: Python, pip, Git, Flask, Boto3, Gunicorn, and AWS CLI Installation on Amazon EC2

Key Components Installed:

- Python 3
- pip Package Manager
- Git
- AWS CLI
- Flask Framework
- Boto3 SDK
- Gunicorn Web Server

5.2 Flask Application Deployment

The Digital Library application was cloned to the EC2 instance and configured with AWS credentials. The Flask application was executed successfully and became accessible through the EC2 public IP address.

```
pip 21.0.1 view /usr/lib/python3.9/site-packages/pip (python 3.9)
git version 2.38.1
[ec2-user@ip-10-0-1-112 ~]$ pip3 install flask boto3 gunicorn
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: flask in ./local/lib/python3.9/site-packages (3.1.3)
Requirement already satisfied: boto3 in ./local/lib/python3.9/site-packages (1.42.97)
Requirement already satisfied: gunicorn in ./local/lib/python3.9/site-packages (23.0.0)
Requirement already satisfied: itsdangerous<=2.2.0 in ./local/lib/python3.9/site-packages (from flask) (2.2.0)
Requirement already satisfied: blinker<=1.9.0 in ./local/lib/python3.9/site-packages (from flask) (1.9.0)
Requirement already satisfied: werkzeug<=3.1.0 in ./local/lib/python3.9/site-packages (from flask) (3.1.0)
Requirement already satisfied: importlib-metadata<=3.6.0 in ./local/lib/python3.9/site-packages (from flask) (8.7.1)
Requirement already satisfied: click<=8.1.3 in ./local/lib/python3.9/site-packages (from flask) (8.1.0)
Requirement already satisfied: markupsafe<=2.1.1 in ./local/lib/python3.9/site-packages (from flask) (3.0.3)
Requirement already satisfied: Jinja2<=3.1.2 in ./local/lib/python3.9/site-packages (from flask) (3.1.0)
Requirement already satisfied: s3transfer<0.17.0, >=0.16.0 in ./local/lib/python3.9/site-packages (from boto3) (0.16.1)
Requirement already satisfied: jmespath<2.0.0, >=0.7.1 in /usr/lib/python3.9/site-packages (from boto3) (0.10.0)
Requirement already satisfied: botocore<1.43.0, >=1.42.97 in ./local/lib/python3.9/site-packages (from boto3) (1.42.97)
Requirement already satisfied: packaging in ./local/lib/python3.9/site-packages (from gunicorn) (26.2)
Requirement already satisfied: python-dateutil<3.0.0, >=2.1 in /usr/lib/python3.9/site-packages (from botocore<1.43.0, >=1.42.97->boto3) (2.8.1)
Requirement already satisfied: urllib3<1.27, >=1.25.4 in /usr/lib/python3.9/site-packages (from botocore<1.43.0, >=1.42.97->boto3) (1.25.10)
Requirement already satisfied: zipp<=3.20 in ./local/lib/python3.9/site-packages (from importlib-metadata<=3.6.0->flask) (3.23.1)
Requirement already satisfied: six<=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil<3.0.0, >=2.1->botocore<1.43.0, >=1.42.97->boto3) (1.15.0)
[ec2-user@ip-10-0-1-112 ~]$ pip3 list
Package Version
-----
attrs 20.3.0
aws-cfn-bootstrap 2.0
awscli 2.33.15
awsert 0.31.1
babel 2.9.1
```

Figure 5.2: Running the Flask Digital Library Application on Amazon EC2

KeySteps:

- Configured AWS credentials
- Executed the Flask application
- Verified successful application startup
- Accessed the application through the browser

5.3 Integration with Amazon S3

Amazon S3 is used to store PDF books uploaded through the Flask application. The application uploads book files to the S3 bucket while maintaining secure access.

Key Functions:

- Upload PDF books
- Store files securely
- Retrieve books for download
- High durability and scalability

5.4IntegrationwithAmazonDynamoDB

Amazon DynamoDB stores metadata for uploaded books, including book ID, title, author, category, upload date, and S3 object key.

Key Functions:

- Store book metadata
- Fast retrieval of records
- On-demand scaling
- NoSQL database support

5.5 Monitoring with Amazon CloudWatch

Amazon CloudWatch is used to monitor the EC2 instance and application performance. It provides metrics and logs that help identify issues and ensure smooth operation.

Key Functions:

- Monitor EC2 performance
- Collect system metrics
- Track application logs
- Support troubleshooting

6.RESULTS

The implementation of the Cloud-Based Digital Library System was successfully completed using AWS cloud services. The system was tested to verify its functionality, reliability, and performance. The Flask web application was deployed on an Amazon EC2 instance and integrated with Amazon S3 and Amazon DynamoDB. The results demonstrate that the system provides secure storage, efficient metadata management, and easy access to digital books.

6.1 Successful Deployment of the Web Application

The Flask-based Digital Library application was successfully deployed on an Amazon EC2 instance. Users can access the application through the EC2 public IP address using a web browser. The application provides a simple interface for uploading and managing digital books.

Results:

- Flask application deployed successfully.
- Application accessible through EC2 Public IP.
- Stable and responsive web interface.
- Successful communication with AWS services.

6.2 Secure Storage of Books using Amazon S3

Amazon S3 successfully stores uploaded PDF books in the configured bucket. The uploaded files are organized efficiently and can be retrieved whenever required.

Results:

- Books uploaded successfully to S3.
- Secure cloud storage.
- Reliable file retrieval.
- High durability and availability.

6.3 Metadata Storage using Amazon DynamoDB

Book information including BookID, Title, Author, Category, Upload Date, and S3 Object Key is successfully stored in the DynamoDB table. The metadata can be retrieved quickly whenever needed.

Results:

- Book metadata stored successfully.
- Fast retrieval of records.
- NoSQL database performance verified.
- On-demand scaling supported.

6.4 Secure Application Hosting using Amazon EC2

The application was successfully hosted on an Amazon EC2 instance inside a custom VPC. Security Groups allowed only the required ports such as SSH, HTTP, HTTPS, and Flask application port.

Results:

- EC2 instance running successfully.
- Public IP enabled application access.
- Secure network configuration.
- Stable application hosting.

6.5 Monitoring using Amazon CloudWatch

Amazon CloudWatch was configured to monitor the EC2 instance. Performance metrics such as CPU utilization, network activity, and system health help monitor the application and improve reliability.

Results:

- EC2 monitoring enabled.
- Resource utilization tracked.
- Performance metrics available.
- Easier troubleshooting and maintenance.

6.6 Overall System Performance

The Digital Library System successfully integrates multiple AWS services to provide an efficient cloud-based solution for storing and managing educational resources. Testing confirmed that book uploads, metadata storage, and application access worked correctly without major issues.

The implemented system provides:

- Secure storage of digital books.
- Fast retrieval of metadata.
- Reliable cloud hosting.
- Scalable cloud infrastructure.
- Easy access through a web browser.

6.7 Screenshots of the Output

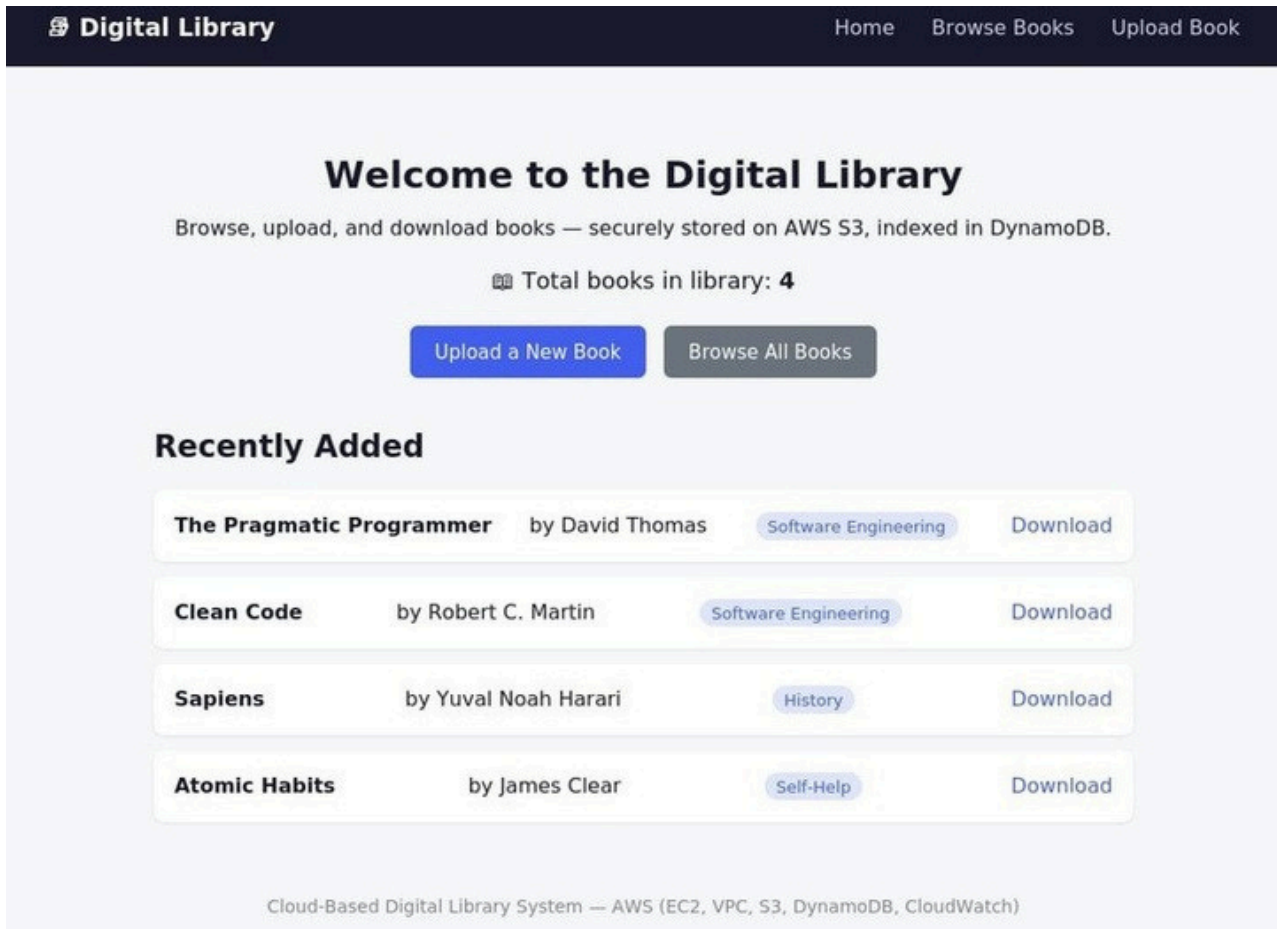


Figure 6.1: Home Page of the Cloud-Based Digital Library System.

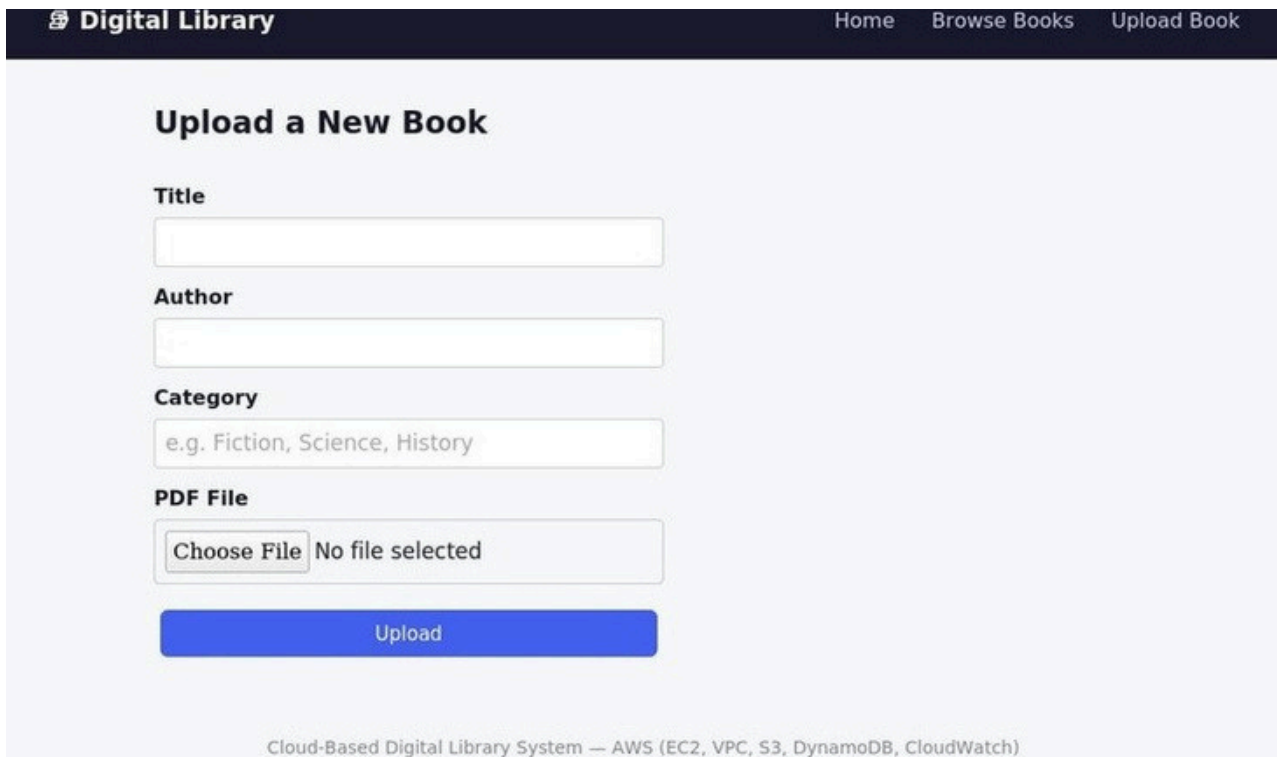


Figure 6.2: Upload Book Interface.

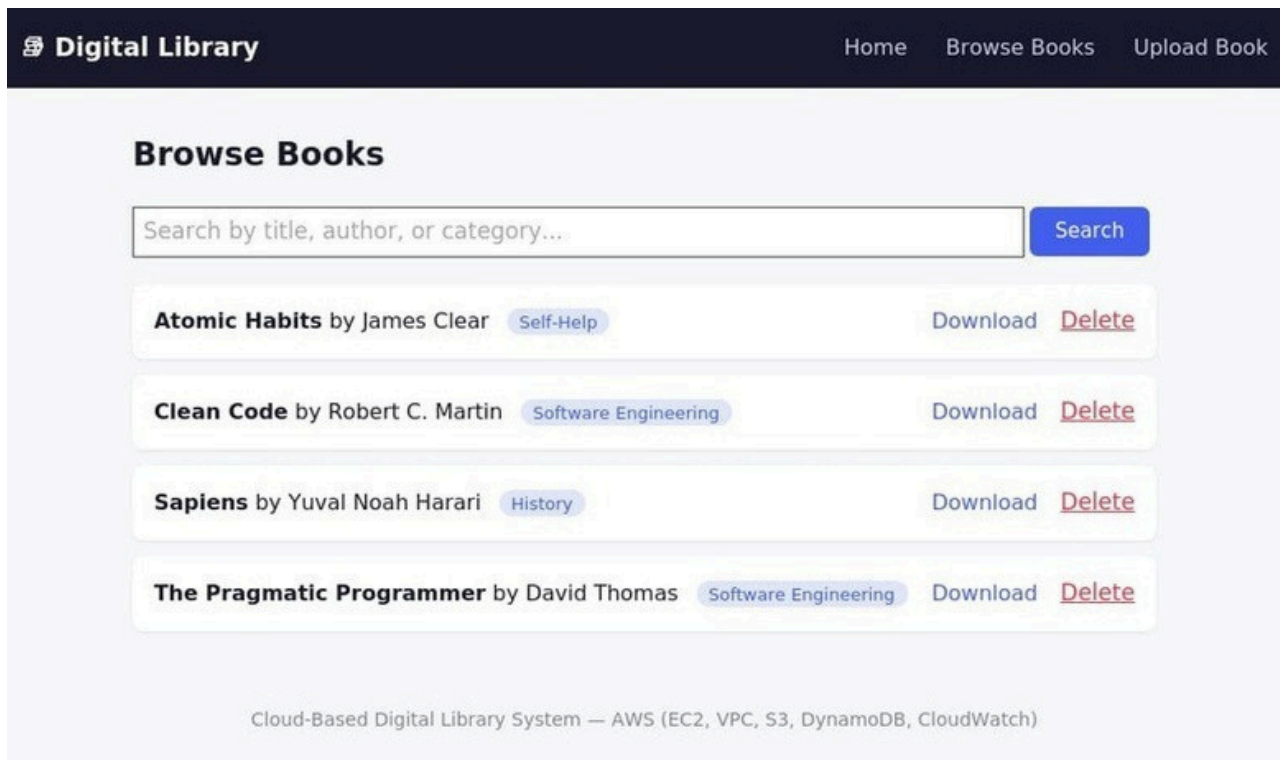


Figure 6.3: Browse Books Page Displaying Available Digital Books.

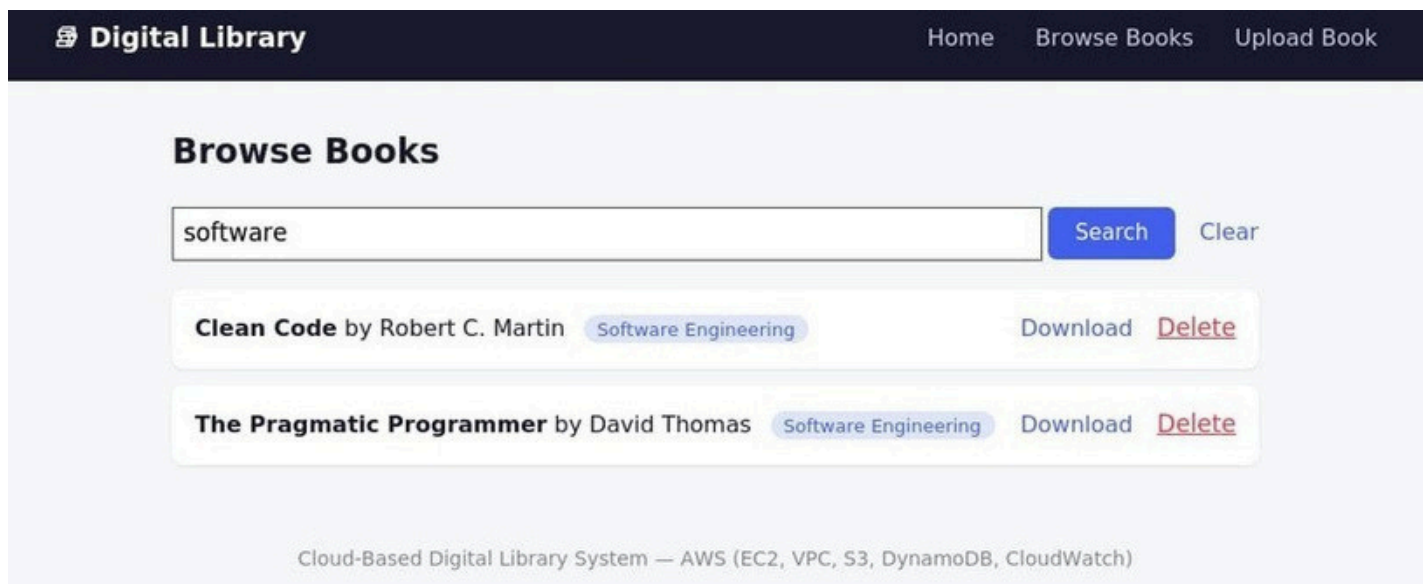


Figure 6.4: Search Functionality Showing Filtered Book Results

7.CONCLUSION

The successful development and deployment of the **Cloud-Based Digital Library System** demonstrate the effective use of AWS cloud services for managing digital books. By integrating **Amazon EC2, Amazon VPC, Amazon S3, Amazon DynamoDB, Amazon CloudWatch, and Flask**, the system provides a secure, reliable, and scalable platform for storing, managing, and accessing digital library resources.

The application enables users to upload PDF books, store them securely in Amazon S3, maintain book metadata in Amazon DynamoDB, and search, browse, download, or delete books through a user-friendly web interface. Hosting the application on Amazon EC2 within a custom VPC ensures secure and reliable access, while Amazon CloudWatch helps monitor the application's performance and resource utilization.

The project highlights the advantages of cloud computing, including high availability, scalability, secure storage, and simplified resource management. Testing confirmed that all major functionalities, including book upload, search, browsing, downloading, and metadata management, operated successfully and met the project objectives.

This project also provides a strong foundation for future enhancements such as:

- User authentication and role-based access control.
- Book recommendations and advanced search filters.
- Integration with Amazon RDS or Elasticsearch for enhanced search capabilities.
- Mobile application support and REST API integration.
-

In conclusion, the **Cloud-Based Digital Library System** successfully demonstrates how AWS cloud technologies can be integrated to build an efficient, secure, and scalable digital library application. The project offers a practical solution for educational institutions and organizations that require reliable cloud-based digital resource management while remaining flexible for future expansion.

8.REFERENCES

- [1] Amazon Web Services, "Amazon EC2 Documentation." Available: <https://docs.aws.amazon.com/ec2/>
- [2] Amazon Web Services, "Amazon S3 Documentation." Available: <https://docs.aws.amazon.com/s3/>
- [3] Amazon Web Services, "Amazon DynamoDB Documentation." Available: <https://docs.aws.amazon.com/dynamodb/>
- [4] Amazon Web Services, "Amazon VPC Documentation." Available: <https://docs.aws.amazon.com/vpc/>
- [5] Amazon Web Services, "Amazon CloudWatch Documentation." Available: <https://docs.aws.amazon.com/cloudwatch/>
- [6] Amazon Web Services, "AWS Security Groups." Available: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
- [7] Pallets Projects, "Flask Documentation." Available: <https://flask.palletsprojects.com/>
- [8] Python Software Foundation, "Python 3 Documentation." Available: <https://docs.python.org/3/>
- [9] Boto3 Documentation, "AWS SDK for Python (Boto3)." Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>
- [10] Git Documentation, "Git Reference Manual." Available: <https://git-scm.com/doc>